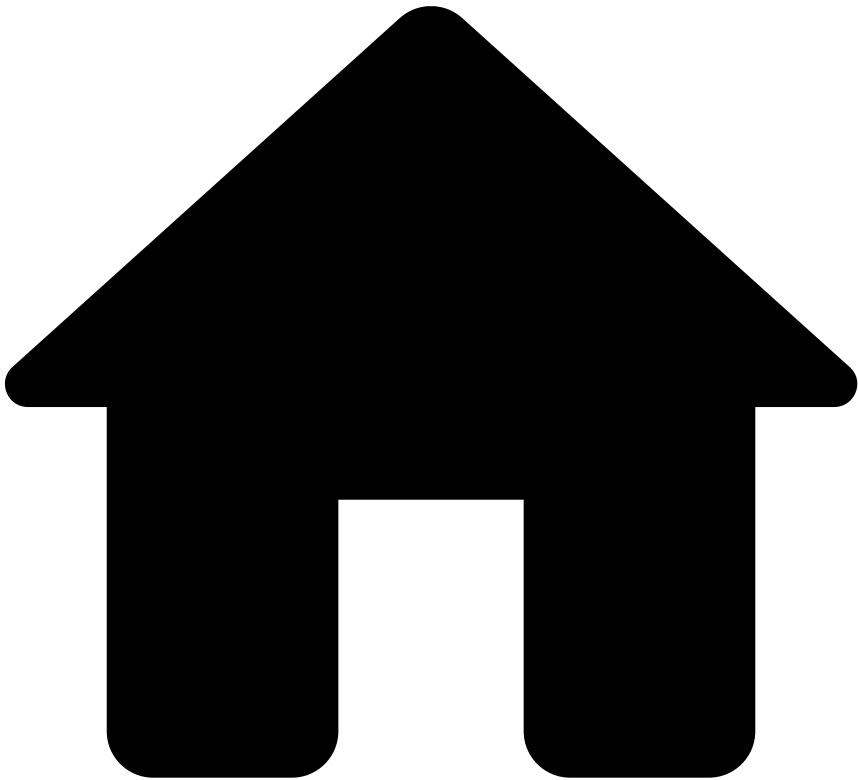


•



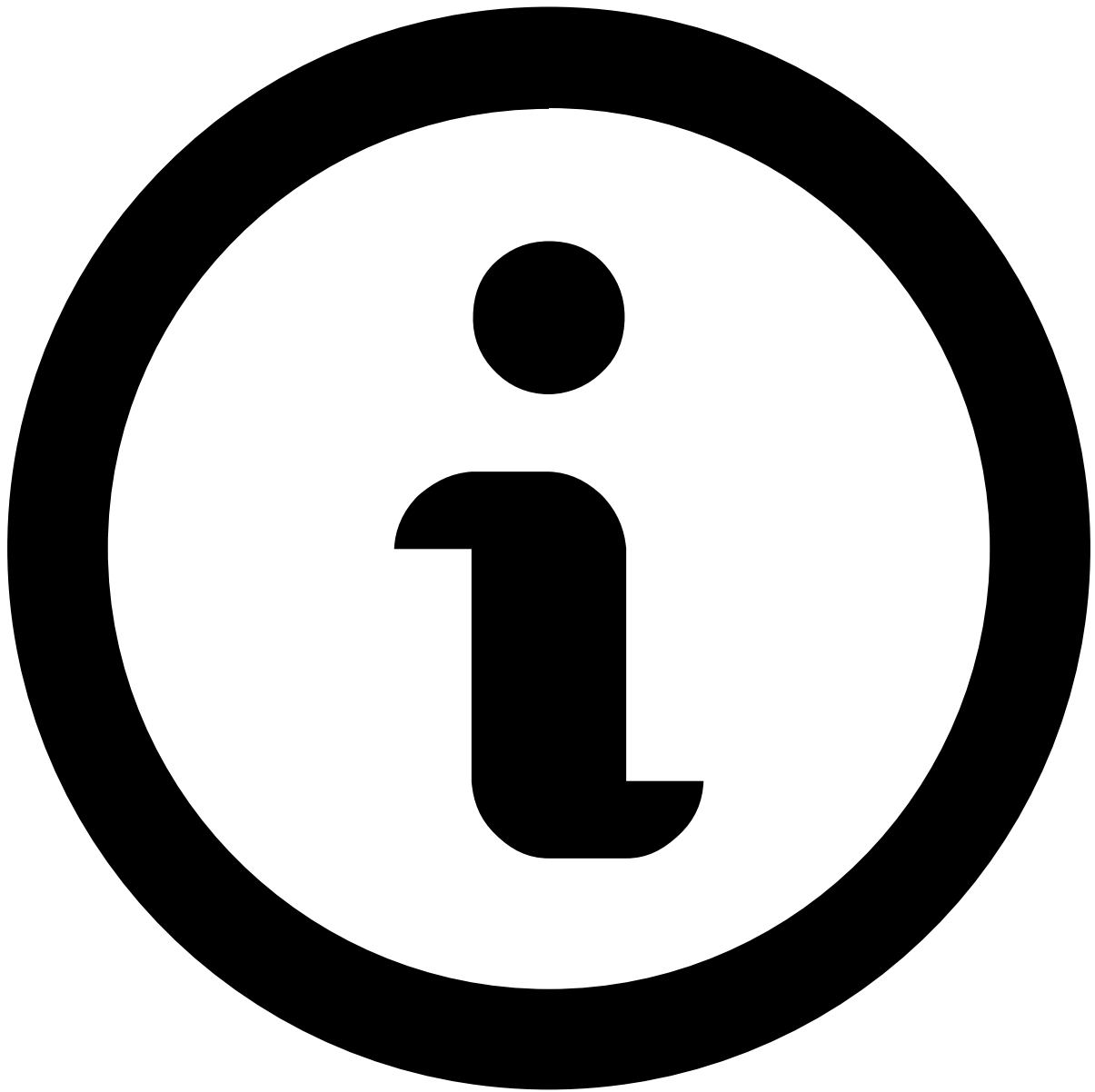
- Best Practices
- [AutoML](#)
- Feature Engineering Stage

On this page

Was this page helpful? 

Best Practices for AutoML's Feature Engineering Stage

At the start of a project, the schema is usually already defined. In addition, fraud or money laundering patterns should also be characterized and understood so that you know which fields to use in the transformations. The transformations already encode Feedzai's knowledge into what is the best strategy to build features to tackle financial crime. This is all that is needed for AutoML basics: you can now create a semantic file with such knowledge in your hand.



Schema definition

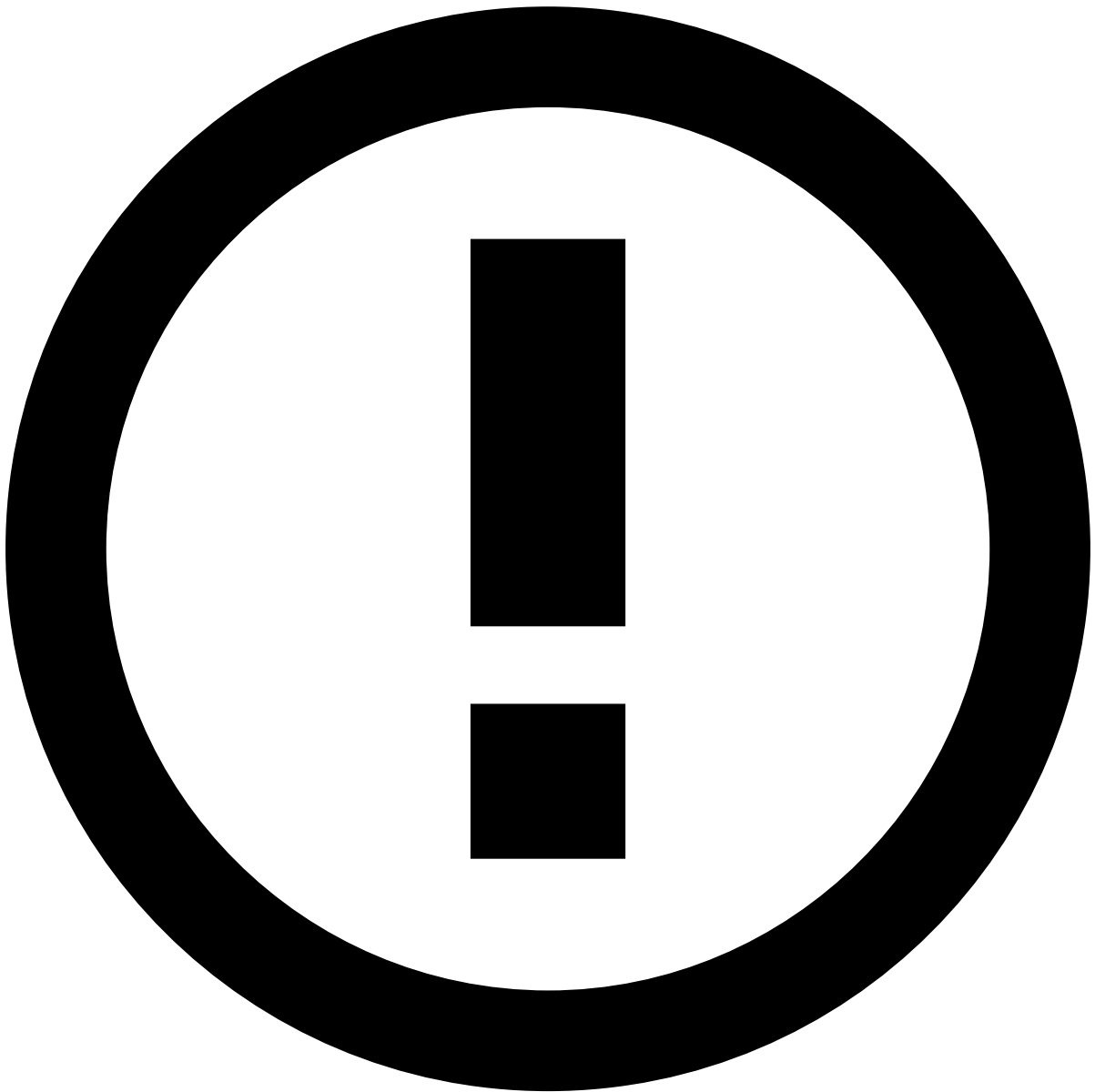
Sometimes it's hard to have schemas completely defined at the beginning of a project. The same applies to the definition of scorable events, and other important aspects of the underlying data generation process that we do not control.

Take this opportunity to reflect on what's the minimum reliable information you have about the data. With it, consider building the semantic file with the least assumptions possible. Start by selecting only one grouping entity and tagging the most important fields, e.g., event timestamp, and transaction amount.

Semantic files let you map schema fields to common entities that are useful for feature engineering. Doing this allows you to speed up the initial task of creating a feature engineering plan from nothing. So, in case you don't have any data yet, you can still use AutoML to create a plan with features that leverage domain knowledge, therefore expected to be useful.

[Learn which tags are supported by semantic files.](#)

When you want to have aggregations with more than one grouping entity, you need to create a *composite field*. This is useful for the case when you want to track behavior over an entity with low cardinality. For example, when processing money transfers, you will most likely want to track how many transactions a person does for a given country. **To do this, you need to define a composite field that combines the originator account and the destination country.**



entity vs grouping-entity tags

Entity tags will only be used for *countDistinct* aggregations. When defining a field as an entity, AutoML will not use it for counts, sums, or any other aggregation that is not a distinct aggregation.

Imagine this scenario for a payment processor's data set: define `customer_id` as grouping-entity and `merchant_country` as an entity. This combination will only create the following subset of profiles. For the sake of simplicity, we're omitting other aggregations based on timestamp (e.g. velocities), or numeric fields (e.g. avg amount).

- `group by customer_id count()`
- `group by customer_id countDistinct(merchant_country)`

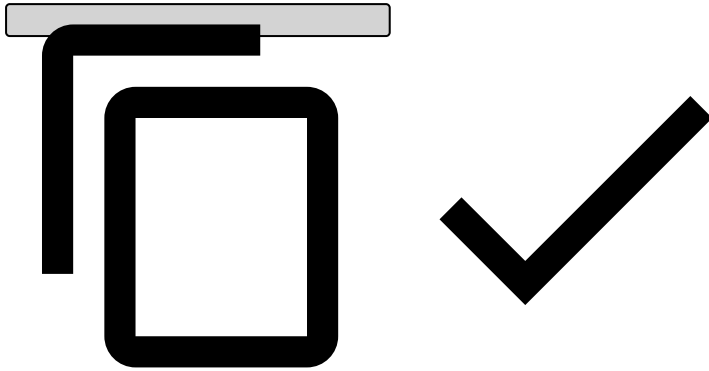
If you also want to track the counts per customer and merchant country, you need to define a composite grouping entity:

```
{
  "@type": "composite",
  "fields": [
    "customer_id",
    "merchant_country"
  ]
}
```

```

],
"tags": [ "grouping-entity" ],
"fieldDesc": "customer_id_merchant_country"
}

```



Keep in mind that grouping entities with very low cardinality (<10) or highly skewed, can compromise your job execution. Check the [Don'ts](#) section below for more context.

Remember that grouping entities will not just create the aggregations/profiles. These metrics will also be used to compute ratios and other derived transformations.

The following is an example of a semantic file.

semantic-file.json

```

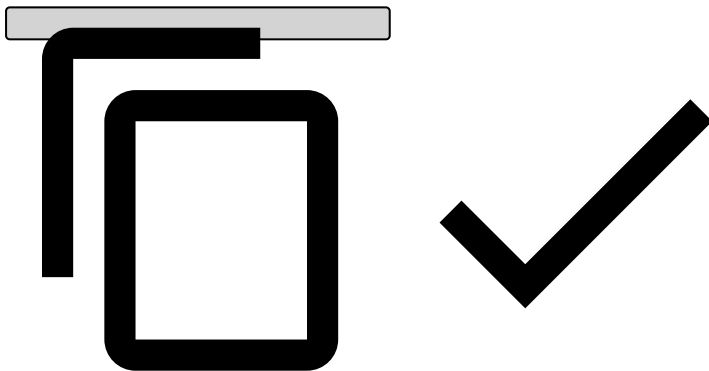
[
  {
    "@type": "simple",
    "tags": [
      "amount"
    ],
    "fieldDesc": "amount"
  },
  {
    "@type": "simple",
    "tags": [
      "entity"
    ],
    "fieldDesc": "acceptor_id"
  },
  {
    "@type": "simple",
    "tags": [
      "entity",
      "country",
      "location"
    ],
    "fieldDesc": "acceptor_country_code"
  },
  {
    "@type": "simple",
    "tags": [
      "timestamp",
      "datetime"
    ],
    "fieldDesc": "event_occurred_at"
  },
  {
    "@type": "simple",
    "tags": [
      "grouping-entity"
    ]
  }
]

```

```

    ],
    "fieldDesc": "card_pan"
  },
  {
    "@type": "simple",
    "tags": [
      "amount"
    ],
    "fieldDesc": "account_available_credit"
  },
  {
    "@type": "simple",
    "tags": [
      "datetime"
    ],
    "fieldDesc": "account_open_date"
  },
  {
    "@type": "simple",
    "tags": [
      "grouping-entity"
    ],
    "fieldDesc": "customer_id"
  }
]

```



Based on the semantic file, AutoML will create a **deployable plan** where it will expand as many features as it can.

Before submitting your AutoML run, be sure to create job profiles with appropriate Spark configurations considering your data volume and complexity. AutoML will use the default job profile of Pulse for each type of job it launches. The only exception is the job profile for feature selection, for which AutoML will use the profile configured in `pulse.properties`. If your AutoML run fails during the feature engineering phase, it's probably due to memory issues. Use job profiles to tune memory usage.

A crucial consideration that affects the feature engineering process is the existence of **partitioning in runtime**. Briefly, runtime partitioning is essential when processing very high throughputs, e.g. 1000 TPS at peak. Having your data partitioned across different machines means that machine 1 will not see **in real time** the data of machine 2. Because of this, you will need to decide what is your main grouping entity and make sure it is the same as the partitioning key. This is usually the customer/card/account field. If you don't match the partitioning keys in the grouping, then AutoML will generate real-time metrics that might not be possible to compute during runtime.

Dosâ

- Verify which timestamp field of your schema is the one that will be used to represent the time in which the event occurred. This field will need to be provided in the first page of the AutoML wizard. **This is the timestamp used to compute the metrics.**
- **Choose one main grouping entity only.** Even if your data is not partitioned in runtime now, consider the likelihood of having partitioning in the future. Remember that when your data is partitioned you won't be able to

leverage entities that can be seen across multiple runtime partitions. On the other hand, more grouping entities lead to a more thorough exploration of feature combinations.

Putting the runtime partitioning issue aside, also consider the pros/cons of having potential marginal gains on model performance versus strain on processing pipeline. Think about what behavior mostly describes your fraud or money laundering patterns. Typical grouping entities are: customer/account, or card (in a card channel).

- Create a job profile that is reasonable for your feature engineering plan.
- If you expect very large data volumes (1M events per day or more), tweak window sizes under the advanced options. As a general rule, avoid real-time windows greater than 1 day: real-time windows are kept in memory, and thus impact memory performance as well as recovery. Read more about this under the [Metrics best practices](#) pages.
- When starting from a Solution (e. g. Transaction Fraud for Retail Banking Cards or Transaction Fraud for Retail Banking Transfers and Instant Transfers), always check first its template semantic file.

Don'ts🚫🚫

- **Avoid grouping entities with very low cardinality** (i.e. low number of different values). This will depend on the project so be aware of this when reusing semantic files. For example, if you're looking at data mostly from 1 or 2 countries, using the country code as a grouping entity can be problematic. All your data will be split per these categories, and your pipeline will run in a very large chunk of data. This in turn will cause out-of-memory errors. However, if you're expecting your data to be well distributed across many countries **and you don't have partitioning in runtime**, this could be a reasonable **grouping** entity.
- Be mindful of your `Entity` tags. Entities are used for distinct counts (e.g. distinct beneficiaries per customer). Too many distinct aggregations on long windows in large data sets is known to have a high memory footprint in the Data Science environment. In addition, these aggregations are very computation-intensive in runtime. As a rule of thumb, try to keep the entities to the minimum needed for the project you are doing.

Data split stage🚫🚫

The data source splitting stage is a sub-stage in the feature engineering stage. At the data split stage, Pulse will split your data set into two or three subsets: training, validation, and optionally testing. Although this intermediate stage is rarely a point of failure, it still deserves some considerations.

As a general Machine Learning guideline, these splits are 80/20/20, i.e., 80% of your data for training, 20% for validation, and 20% for testing (hold-out data set). Here, we enforce that the split be defined timewise, because our transformations are time sensitive.

Splitting is done through a filter, which can generate data with empty partitions (or blocks) for the validation and testing splits. This doesn't impact the model results. However, Data Scientists might want to use these validation and test sets for other operations. In this case, they will experience degraded computational performance compared to the same data with these empty blocks removed. In the long run, having empty data partitions will cause HDFS/S3 degradation related to having many empty files/blocks.

To solve this, manually partition your data through JupyterLab by coalescing the data source. As a rule of thumb, if the data source is 20% of the initial data, it should have 20% the number of partitions (see the snippet below).

```
>>> # Load the validation set through the ds-api
>>> df_valid = feedzai.pulse_to_pyspark(
...     spark_session, app_name, datasource_name
... )
>>> # The number of partitions will be the same as in the feature eng data source
>>> df_valid.rdd.getNumPartitions()
1000
>>> feedzai.pyspark_to_pulse(
...     spark_session,
...     # Save a new data source coalesced to have only 200 partitions
...     df_valid.coalesce(200),
...     app_name,
```

```
...     datasource_name + "_coalesced",  
...     "/path/for/this/new/datasource",  
... )
```

